EXP 10: Automating Application Deployment Using a CI/CD Pipeline

Theory:
Serverless functions let developers code in any language or framework with which they're comfortable. Simpler backend code. Serverless removes a lot of coding complexity for developers, allowing them to create simple, self-contained functions that independently perform one purpose.

A continuous integration and continuous deployment (CI/CD) pipeline is a series of steps that must be performed in order to deliver a new version of software. CI/CD pipelines are a practice focused on improving software delivery throughout the software development life cycle via automation.

AWS IAM - AWS Identity and Access Management (IAM) is a web service for securely controlling access to AWS services. With IAM, you can centrally manage users, security credentials such as access keys, and permissions that control which AWS resources users and applications can access.

AWS CodeDeploy is a service that automates code deployments to any instance, including Amazon EC2 instances and instances running on-premises. AWS CodeDeploy makes it easier for you to rapidly release new features, helps you avoid downtime during deployment, and handles the complexity of updating your applications.

Steps for Execution:

Step 1: Create IAM Role for EC2
- Go to **IAM (Identity and Access Management)** in AWS Console.
- Click **Roles → Create Role**.
- Select:
  - Trusted entity: **AWS Service**
  - Use case: **EC2**
- Click **Next**.

Step 2: Attach Permissions
- Add permission:
  - **AmazonS3ReadOnlyAccess**
- Click **Next → Create Role**.

Step 3: Create Role for CodeDeploy
- Again click **Create Role**.
- Select:
  - Trusted entity: **AWS Service**
  - Use case: **CodeDeploy**
- Attach policy:
  - **AWSCodeDeployRole**
- Create the role.

Step 4: Launch EC2 Instance
- Go to **EC2 Dashboard**.
- Launch a **Linux instance (Amazon Linux AMI)**.
- Connect to the instance terminal.
- Switch to root user:

```
sudo su
```

Step 5: Update System
- Run:

```
sudo yum update
```

Step 6: Install Required Software
- Install Ruby:

sudo yum install ruby

- Install wget:

  sudo yum install wget

Step 7: Install CodeDeploy Agent

- Download installer:

  wget https://aws-codedeploy-us-east-1.s3.amazonaws.com/latest/install
- Make file executable:

  chmod +x ./install
- Install agent:

  sudo ./install auto
- Check status:

  sudo service codedeploy-agent status

Step 8: Attach IAM Role to EC2

- Go to EC2 → Select instance.
- Click **Actions → Security → Modify IAM Role**.
- Attach the created IAM Role.

Step 9: Configure Security Group

- Go to **Security Groups**.
- Edit **Inbound Rules**:
- Add port (e.g., **4000**)
- Source: **Anywhere (IPv4)**
- Save rules.

Step 10: Create Directory in EC2

- Exit root:

  exit
- Create folder:

  mkdir server

Step 11: Create CodePipeline

- Go to **AWS CodePipeline**.
- Click **Create Pipeline**.

Step 12: Connect GitHub Repository

- Choose **GitHub** as source.
- Login and connect repository.
- Select repository and branch.

Step 13: Configure Build Stage (Optional)

- Use **CodeBuild** if needed.
- Provide **buildspec.yaml** file.

Step 14: Create CodeDeploy Application

- Go to **CodeDeploy**.
- Create **Application**.
- Create **Deployment Group**:
- Select EC2 instances
- Use tags (key & value)
- Disable load balancer

Step 15: Add Deploy Stage

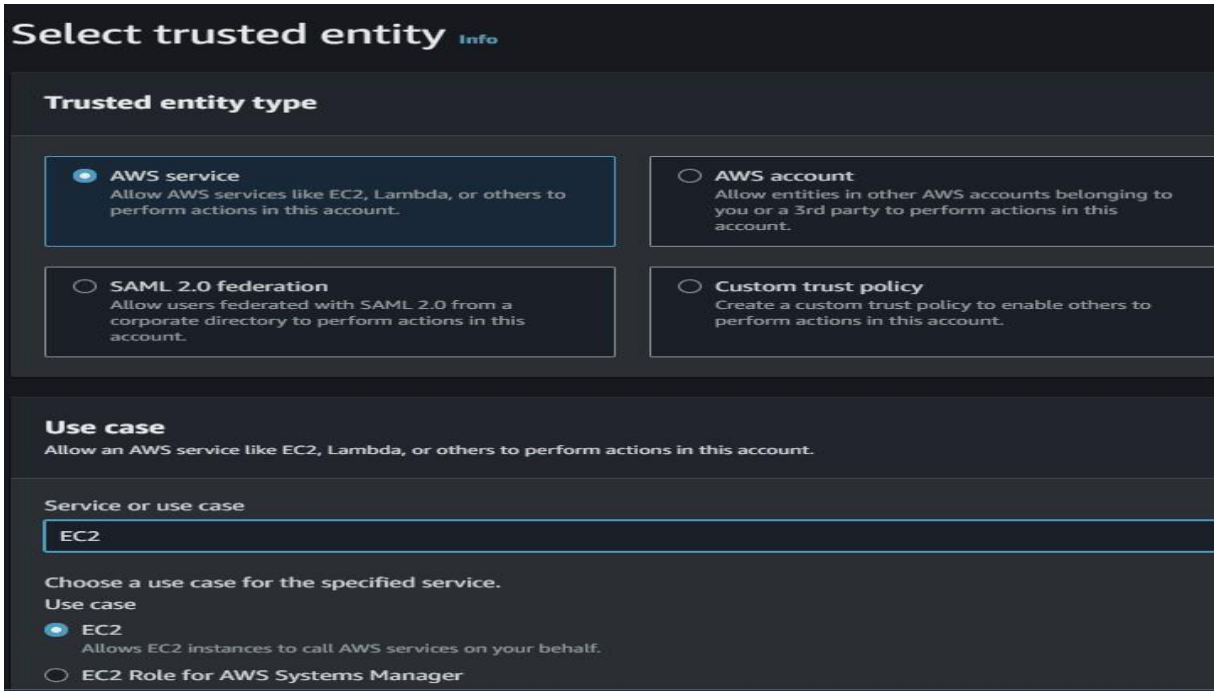
- Return to **CodePipeline**.

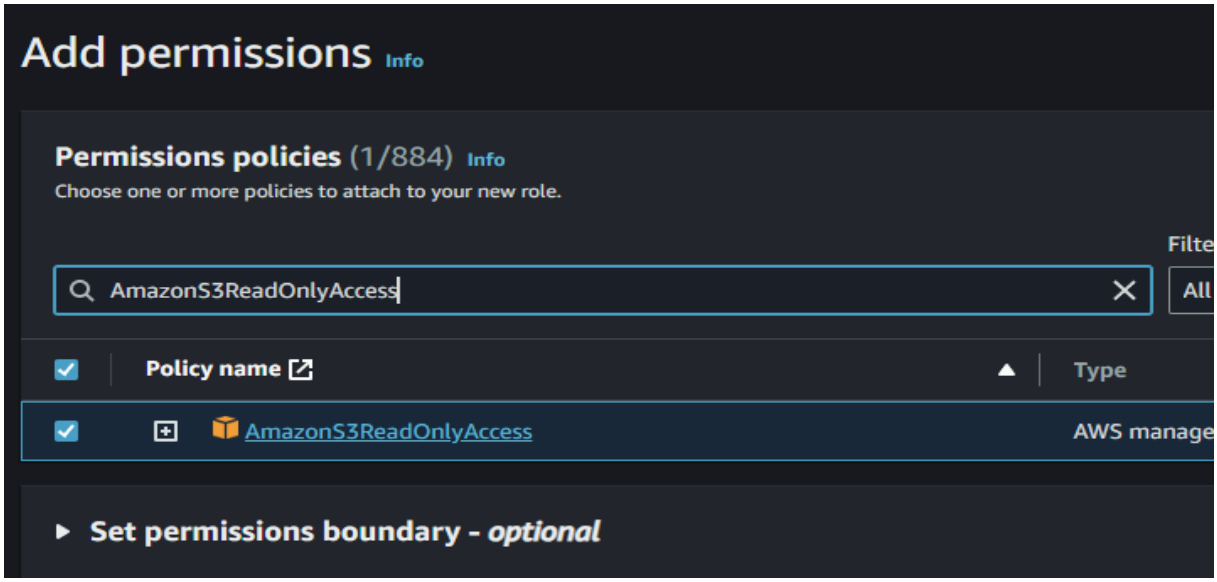
- Add deploy stage:
- Select CodeDeploy application
- Select deployment group

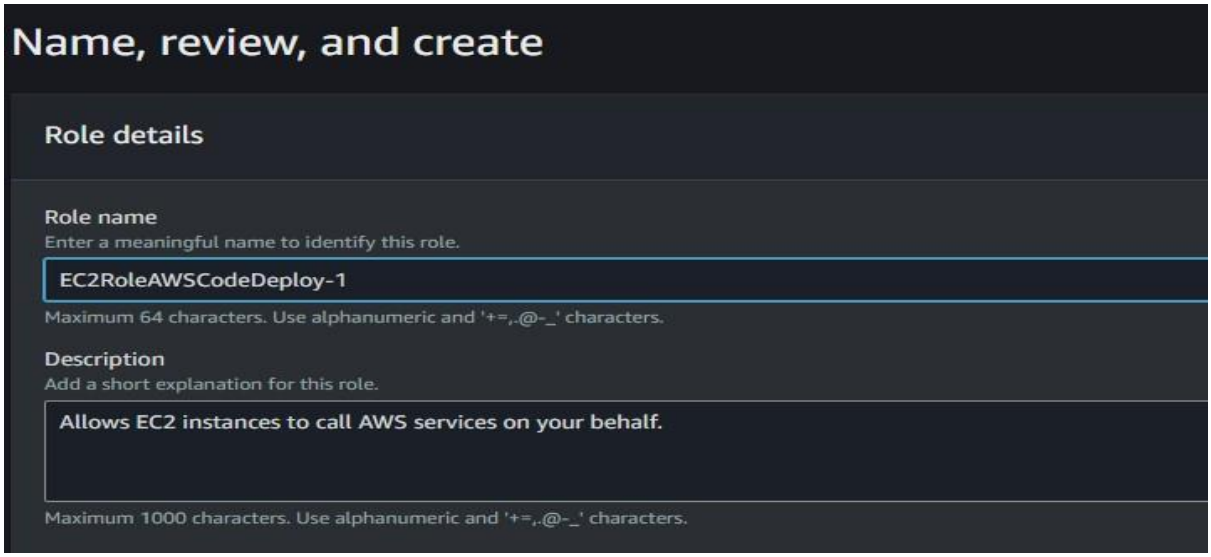
Step 16: Review and Create Pipeline

- Review all configurations.
- Click **Create Pipeline**.

EXP 10: Automating Application Deployment Using a CI/CD Pipeline







Select trusted entity Info

Trusted entity type

- (selected) **AWS service**
  Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- **AWS account**
  Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- **SAML 2.0 federation**
  Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- **Custom trust policy**
  Create a custom trust policy to enable others to perform actions in this account.

Use case

Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

CodeDeploy

Choose a use case for the specified service.

Use case

- (selected) **CodeDeploy**
  Allows CodeDeploy to call AWS services such as Auto Scaling on your behalf.
- **CodeDeploy for Lambda**
  Allows CodeDeploy to route traffic to a new version of an AWS Lambda function version on your behalf.
- **CodeDeploy - ECS**
  Allows CodeDeploy to read S3 objects, invoke Lambda functions, publish to SNS topics, and update ECS services on your be

Add permissions Info

Permissions policies (1) Info

The type of role that you selected requires the following policy.

Policy name

AWSCodeDeployRole

▸ **Set permissions boundary - *optional***

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

AWSCodeDeployRole

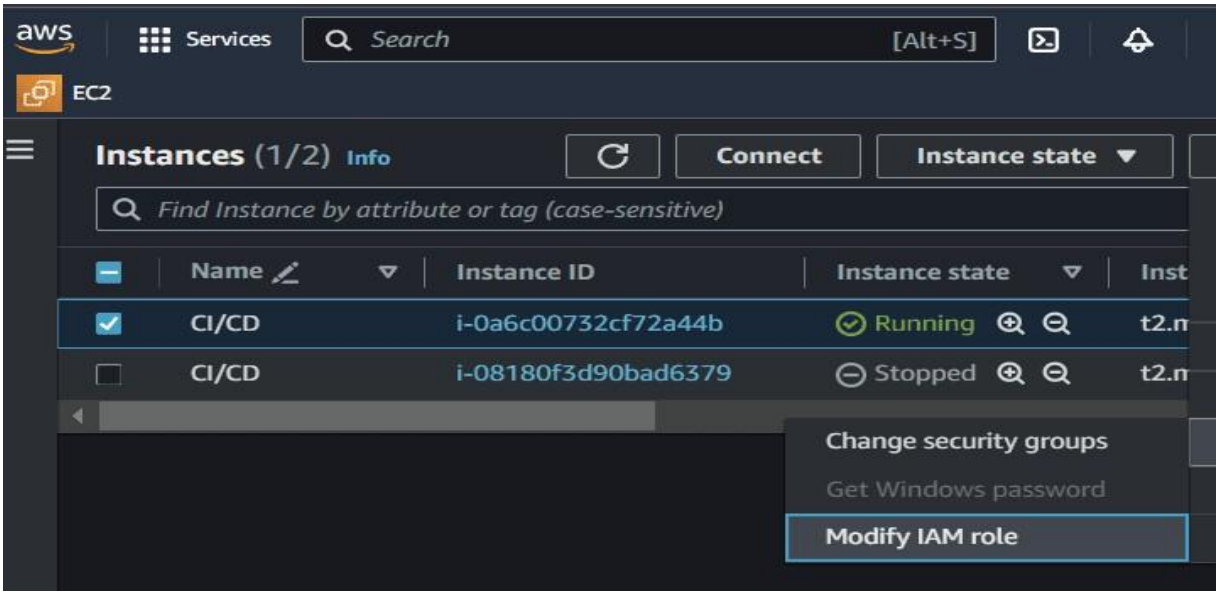
Maximum 64 characters. Use alphanumeric and '+=,.@-_' characters.

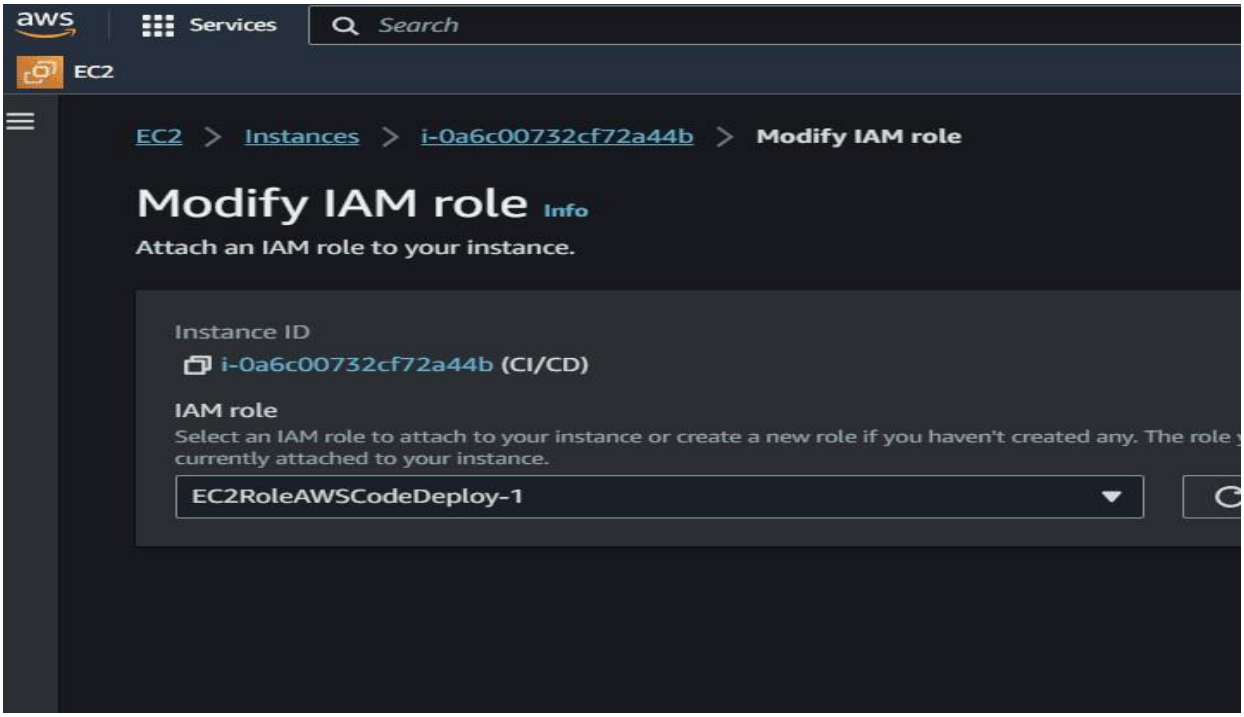
Description

Add a short explanation for this role.

Allows CodeDeploy to call AWS services such as Auto Scaling on your behalf.

Maximum 1000 characters. Use alphanumeric and '+=,.@-_' characters.


aws
Services
Search
[Alt+S]
EC2
Instances (1/2) Info
Connect
Instance state
Find Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
CI/CD
i-0a6c00732cf72a44b
Running
CI/CD
i-08180f3d90bad6379
Stopped
Change security groups
Get Windows password
Modify IAM role


aws
Services
Search
EC2
EC2 > Instances > i-0a6c00732cf72a44b > Modify IAM role
Modify IAM role Info
Attach an IAM role to your instance.
Instance ID
i-0a6c00732cf72a44b (CI/CD)
IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role
currently attached to your instance.
EC2RoleAWSCodeDeploy-1

aws Services N. Virginia

EC2

```
Complete!
I, [2023-10-22T18:30:38.357273 #30431]  INFO -- : Update check complete.
I, [2023-10-22T18:30:38.357450 #30431]  INFO -- : Stopping updater.
[root@ip-172-31-27-184 ec2-user]# sudo service codedeploy-agent status
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modul
_format.rb:37: warning: Passing safe_level with the 2nd argument of ERB.
ted. Do not use it, and specify other arguments as keyword arguments.
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modul
_format.rb:37: warning: Passing trim_mode with the 3rd argument of ERB.n
ed. Use keyword argument like ERB.new(str, trim_mode: ...) instead.
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modul
p_format.rb:27: warning: Passing safe_level with the 2nd argument of ERB
ated. Do not use it, and specify other arguments as keyword arguments.
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modul
p_format.rb:27: warning: Passing trim_mode with the 3rd argument of ERB.
ted. Use keyword argument like ERB.new(str, trim_mode: ...) instead.
The AWS CodeDeploy agent is running as PID 30588
[root@ip-172-31-27-184 ec2-user]# ls
install
[root@ip-172-31-27-184 ec2-user]# exit
exit
[ec2-user@ip-172-31-27-184 ~]$ ls
install
[ec2-user@ip-172-31-27-184 ~]$ mkdir server
[ec2-user@ip-172-31-27-184 ~]$ ls
install  server
[ec2-user@ip-172-31-27-184 ~]$
```

i-0a6c00732cf72a44b (CI/CD)

PublicIPs: 54.163.18.150 PrivateIPs: 172.31.27.184

aws Services Search [Alt+S]

EC2

EC2 > Security Groups > sg-0350c4be16ef18ce1 - launch-wizard-2 > Edit inbound rule

Edit inbound rules Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules Info

Inbound rule 1

Security group rule ID
sgr-07b0a1e0f832560bc

Type Info
Custom TCP

Protocol
TCP

Port range Info
4000

Source type Info
Custom

Source
0.0.0.0/

Description - optional Info

Developer Tools > CodePipeline > Pipelines > Create new pipeline

Step 1
Choose pipeline settings

Step 2
Add source stage

Step 3
Add build stage

Step 4
Add deploy stage

Step 5
Review

Choose pipeline settings Info

Pipeline settings

Pipeline name
Enter the pipeline name. You cannot edit the pipeline name after it is created
reactDemoGithubConnection-1
No more than 100 characters

Service role
New service role
Create a service role in your account

Role name
AWSCodePipelineServiceRole-us-east-1-reactDemoGithubConne
Type your service role name

Allow AWS CodePipeline to create a service role so it can be us
pipeline

Advanced settings

Step 2
Add source stage
Step 3
Add build stage
Step 4
Add deploy stage
Step 5
Review

Source

Source provider
This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.
GitHub (Version 2)

New GitHub version 2 (app-based) action
To add a GitHub version 2 action in CodePipeline, you create a connection, which uses GitHub App access your repository. Use the options below to choose an existing connection or create a new or more

Connection
Choose an existing connection that you have already configured, or create a new one and then return to this task.
arn:aws:codestar-connections:us-east-1:160990010622:connection/9757e0f or Connect t

Ready to connect
Your GitHub connection is ready for use.

Repository name
Choose a repository in your GitHub account.
DotUrDesign/codepipelinedemo
You can type or paste the group path to any project that the provided credentials can access. Use the format 'group/subgroup

Branch name
Choose a branch of the repository.
main

Change detection options
Start the pipeline on source code change
Automatically starts your pipeline when a change occurs in the source code. If turned off, your pipeline only runs if you start it manually or on a schedule.

Output artifact format
Choose the output artifact format.
CodePipeline default
AWS CodePipeline uses the default zip format for artifacts in the pipeline. Does not include Git metadata about the repository.
Full clone
AWS CodePipeline passes metadata about the that allows subsequent actions to do a full Git supported for AWS CodeBuild actions.

Cancel Previous

Developer Tools > CodePipeline > Pipelines > Create new pipeline

Step 1
Choose pipeline settings
Step 2
Add source stage
Step 3
Add build stage
Step 4
Add deploy stage
Step 5
Review

Add build stage Info

Build – *optional*

Build provider
This is the tool of your build project. Provide build artifact details like operating sys
AWS CodeBuild

Region
US East (N. Virginia)

Project name
Choose a build project that you have already created in the AWS CodeBuild console and then return to this task.
reactDemoBuildProject

Environment variables – *optional*
Choose the key, value, and type for your CodeBuild environment variables. In the va CodePipeline. Learn more

Add environment variable

Build type
Single build
Triggers a single build.
Batch build
Triggers multiple buil execution.

EC2

Developer Tools
CodeDeploy

- Source • CodeCommit
- Artifacts • CodeArtifact
- Build • CodeBuild
- Deploy • CodeDeploy
  - Getting started
  - Deployments
  - Applications
  - Deployment configurations
  - On-premises instances
- Pipeline • CodePipeline
- Settings

Go to resource
Feedback

Developer Tools > CodeDeploy > Applications > Create application

Create application

Application configuration

Application name
Enter an application name
ec2App
100 character limit

Compute platform
Choose a compute platform
EC2/On-premises

Tags

Add tag

Cancel

Deployments | Deployment groups | Revisions

Deployment groups

View details | Edit | Create deployme

Name	Status	Last atte

No deployment groups

Before you can deploy your application using CodeDeploy, you must create a deployment group.

Create deployment group

Developer Tools > CodeDeploy > Applications > ec2App > **Create deployment group**

Create deployment group

Application

Application
ec2App
Compute type
EC2/On-premises

Deployment group name

Enter a deployment group name

AppDepGrp

100 character limit

Service role

Enter a service role
Enter a service role with CodeDeploy permissions that grants AWS CodeDeploy access to your target instances.

arn:aws:iam::160990010622:role/AWSCodeDeployRole-1

Deployment type

Choose how to deploy your application

In-place
Updates the instances in the deployment group with the latest application revisions. During a deployment, each instance will be briefly taken offline for its update

Blue/green
Replaces the instances in the deployment group with new instances and deploys the latest application revision to them. After instances in the replacement environment are registered with a load balancer, instances from the original environment are deregistered and can be terminated.

Environment configuration

Select any combination of Amazon EC2 Auto Scaling groups, Amazon EC2 instances, and on-premises instances to add to this deployment

- [ ] Amazon EC2 Auto Scaling groups
- [x] Amazon EC2 instances

  2 unique matched instances. Click here for details

You can add up to three groups of tags for EC2 instances to this deployment group.
One tag group: Any instance identified by the tag group will be deployed to.
Multiple tag groups: Only instances identified by all the tag groups will be deployed to.

Tag group 1

Key	Value - *optional*	
Name	CI/CD	Remove tag

Add tag

+ Add tag group

- [ ] On-premises instances

Matching instances

2 unique matched instances. Click here for details

Agent configuration with AWS Systems Manager Info

Complete the required prerequisites before AWS Systems Manager can install the CodeDeploy Agent. Make sure the AWS Systems Manager Agent is installed on all instances and attach the required IAM policies to them. Learn more

Install AWS CodeDeploy Agent

- ( ) Never
- ( ) Only once
- (•) Now and schedule updates

Agent configuration with AWS Systems Manager Info

Complete the required prerequisites before AWS Systems Manager can install the CodeDeploy Agent. Make sure the AWS Systems Manager Agent is installed on all instances and attach the required IAM policies to them. Learn more

Install AWS CodeDeploy Agent

- ( ) Never
- ( ) Only once
- (•) Now and schedule updates

Basic scheduler | Cron expression

14 | Days

Deployment settings

Deployment configuration

Choose from a list of default and custom deployment configurations. A deployment configuration is a set of rules that determines how fast an application is deployed and the success or failure conditions for a deployment.

CodeDeployDefault.AllAtOnce or Create deployment configuration

Load balancer

Select a load balancer to manage incoming traffic during the deployment process. The load balancer blocks traffic from each instance while it's being deployed to and allows traffic to it again after the deployment succeeds.

- [ ] Enable load balancing

▸ Advanced - optional

Cancel | Create deployment group

Add deploy stage Info

Deploy – *optional*

Deploy provider
Choose how you deploy to instances. Choose the provider, and then provide the configuration details for that provider.

AWS CodeDeploy

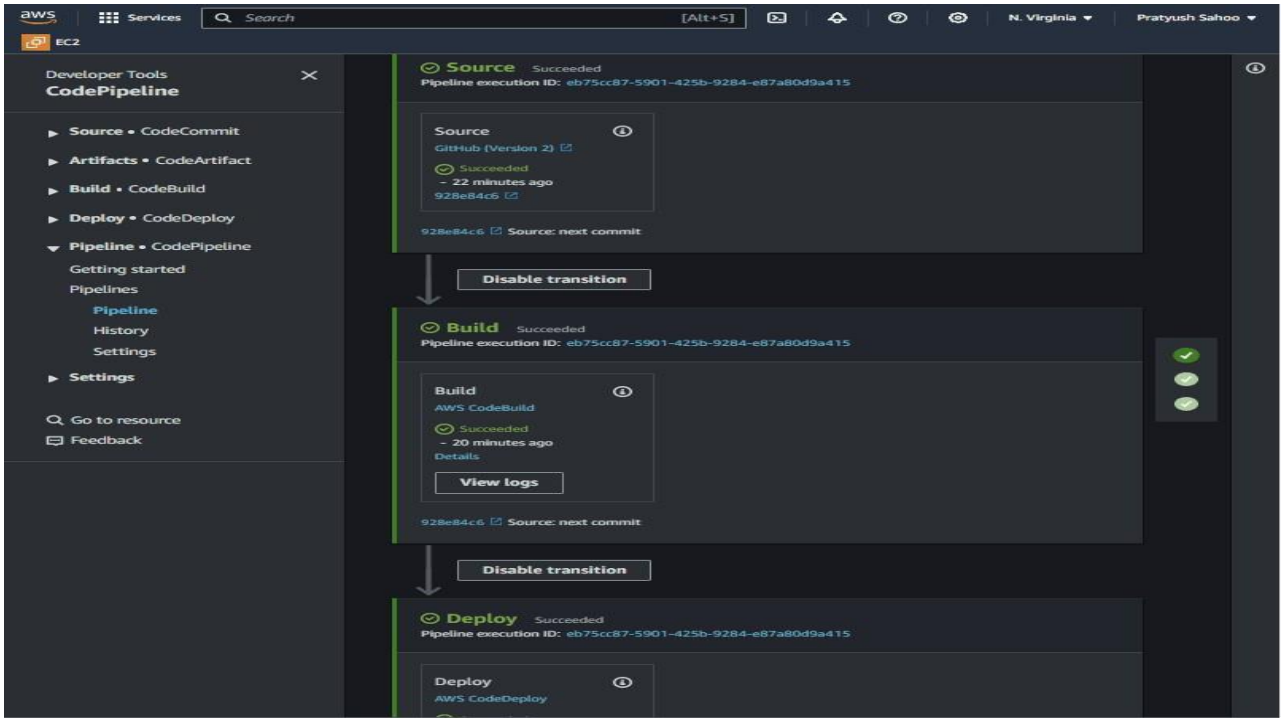
Region

US East (N. Virginia)

Application name
Choose an application that you have already created in the AWS CodeDeploy console. Or create an application in the AWS Cod console and then return to this task.

ec2App

Deployment group
Choose a deployment group that you have already created in the AWS CodeDeploy console. Or create a deployment group in t CodeDeploy console and then return to this task.

AppDepGrp

Cancel | Previous | Skip deploy stage